Swarm Dialogues: Communication policies that shape Embedded Evolution in ESP32 Collectives

By

YALLICO YLQUIMICHE, LUIS ANGEL

MSc Robotics Dissertation





School of Engineering Mathematics and Technology
UNIVERSITY OF BRISTOL
&
School of Engineering
UNIVERSITY OF THE WEST OF ENGLAND

A MSc dissertation submitted to the University of Bristol and the University of the West of England in accordance with the requirements of the degree of MASTER OF SCIENCE IN ROBOTICS in the Faculty of Engineering.

August 28, 2025

Declaration of own work

I declare that the work in this MSc dissertation was carried out in accordance with the requirements of the University's Regulations and Code of Practice for Research Degree Programmes and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, the work is the candidate's own work. Work done in collaboration with, or with the assistance of, others, is indicated as such. Any views expressed in the dissertation are those of the author.

Luis Yallico Ylquimiche on August 28, 2025

Ethics statement

*This project did not require ethical review, as determined by my supervisor **Dr. Paul O'Dowd**.*

Luis Yallico Ylquimiche on August 28, 2025

Swarm Dialogues: Communication policies that shape Embedded Evolution in ESP32 Collectives

Luis Angel Yallico Ylquimiche

***Abstract*—Using a hardware-in-the-loop swarm, we use peer-to-peer ESP-NOW transmissions to quantify the impact of communication policies in swarms (3-13 robots). We compare *stochastic* peer ordering with *comm-aware* scheduling and test two contention controls: per-agent message limits and latency-bounded transmissions. *Comm-aware* scheduling lowers latency by 60%, *stochastic* ordering spreads information faster but increases collision errors by 72% at 13 agents across schemes. Bounded transmission-delays stabilise latency with minimal throughput cost. Design rule: use *stochastic* scheduling with bounded delays for rapid diffusion, use link-aware scheduling when latency/jitter robustness is a priority.**

I. Introduction

This study profiles swarm communication on ESP32 hardware and demonstrates how agent density, locomotion, topology inference, imposed message limits, and transmission policies shape both network quality of service (QoS) and embedded evolution performance. Focusing on direct peer-to-peer links, we contribute with an empirical mapping that probes communication bottlenecks flagged in recent surveys, and evidence for extracting tangible design rules for swarm networks.

Swarm engineers draw inspiration from social biological systems such as ants, bees or termites to build decentralised robot collectives that are inherently robust to failure, flexible across tasks and scalable in number [1]. In swarm systems, collective intelligence emerges when individual robots trade packets of information among neighbouring robots. Classic ant-colony-optimisation work in the early 2000s has already proven that an indirect information exchange of "virtual-pheromones" can lead to agents collectively discovering optimal routing formations [2]. Hence, communication design is a first-order determinant of emergent behavior in swarms.

While coordination and task allocation have been widely studied in swarm robotics, two recent surveys agree that communication bandwidth, latency and energy usage are the main blockers to real-world swarm deployments [3][4]. These issues become more pronounced as swarms sizes scale, resulting in an increase in data volumes being transmitted among peers, often overwhelming individual agents' limited compute capabilities. Beyond sheer capacity, the architecture of the communication also matters. Many swarms rely on blind broadcast communication schemes that scale poorly, with collision rates rising sharply beyond a few dozen peers which tend to reduce the reliability of the network [4].

Communication is equally critical when controllers evolve on-line. Recent embodied evolution studies explain that less communication can enhance swarm performance, as trimming neighbourhood size helps populations forget outdated beliefs and re-adapt faster [5][3]. While our context is controller evolution, we instantiate it as a distributed global optimisation problem. Fixing behaviours and evolving solutions to a known objective, isolates the communication effects and makes adaptation dynamics comparable across conditions.

To support our investigation, we implement a hardware-in-the-loop framework by leveraging over-the-air (OTA) updates. For embedded evolution we use ESP-NOW with an island-model genetic algorithm (GA). Moving beyond simulation-heavy work, we experiment with Pololu 3Pi+ robots, as shown in Fig. 1b, and adopt a state-of-the-art cloud enabled system that captures granular packet and evolution data in real-time, yielding reproducible datasets and enabling controlled parameter sweeps.

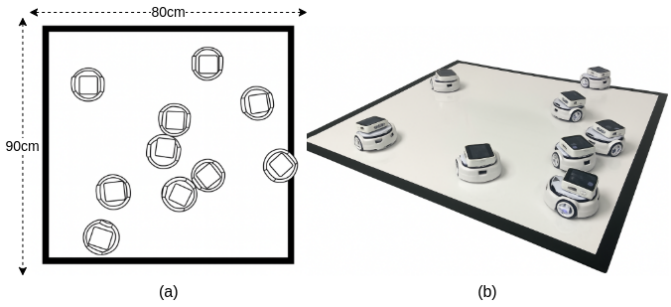


Fig. 1. Experimental setup: (a) arena dimensions and (b) example of a Brownian locomotion experiment with 8 robots.

We explore three communication factors and their measurable effects: (i) peer ordering, (ii) fixed per-robot message limits, and (iii) per-packet bounded transmission delays. We ask, **RQ1**: whether peer ordering using *link-quality* as a lightweight proxy for swarm topology (see Section IV-D1) affects network properties and QoS, **RQ2**: whether these factors change information-diffusion speed and embedded evolution convergence, and **RQ3**: whether per-packet transmission delays can mitigate the impact of collisions during message bursts. Section VI discusses how each RQ shapes two design rules for ESP32 swarms.

II. Related Work

Island-model evolutionary algorithms in [6], show that migration (genome exchange) topology and connectivity structure govern convergence dynamics, not just the migration rate. Rastrigin function Eq. 1, was used to benchmark the performance of different asynchronous evolutionary algorithms.

In [7], controllers are evolved for ad-hoc aerial relays without positional information, using periodic single-hop broadcasts to neighbours. This concept is leveraged on our experimental setting where the agents exploit link-quality proxies to bias their connections. In non-local communication schemes such as [8], network-wide epidemic broadcasting known as *flooding* is used to accelerate the diffusion of information, in environmental mapping. In contrast, our study uses *unicast* peer-to-peer communication, enabling direct round-trip time measurements (latency).

Constrained connectivity repeatedly appears beneficial. The *"less is more"* effect in [9], demonstrates that fewer links and lower swarm densities improve adaptation in swarm consensus tasks, by helping robots discard stale beliefs. A similar effect is also observed by [5] over embodied evolution, where limiting the genome-exchange range can support the evolutionary search maintain higher diversity for longer periods. We explore similar properties, the main difference being the data-link used and our message scheduling algorithm.

Transmission timing also matters. Experiments from [10] show that limiting the frequency of communication stabilises swarm behavior, while [11] explains that the speed of consensus among peers tolerates random transmission delays even when agents receive multiple out-of-order messages. Furthermore, non-radio data links have also been explored by [12], which shows infra-red (IR) line-of-sight local communication can be used in swarms, with benefits such as lower energy requirements per bit. Moreover, with limits on bandwidth and bit error probability that scales with range.

Complementary work by [13] examines real-time communication middleware for swarms at the application layer, addressing peer-to-peer IP protocols. In contrast we explore direct peer-to-peer messaging without any middleware.

III. Experimental Setting

In this study, we use Eq. 1 to benchmark the evolutionary performance of the swarm. This global optimisation task was chosen to emulate evolutionary controller optimisation, while no controller was actually evolved the concept of robots sending and receiving genomes from their peers remains the same, making swarm behaviour a control variable. This design choice allows for a controlled investigation of the effects of message propagation, without confounding results with adaptive changes in agent policy.

The environment for the experiments was a rectangular arena measuring 80x90cm without obstacles (Fig. 1a), where the initial positions, and number of the robots was determined by the experiment schedule (Table IV). Across the study, we evaluated swarm under different communication and system configurations. Each experiment manipulates a specific independent variable while holding all other conditions constant, these include:

- **Swarm density**: The number of agents deployed simultaneously, ranging from 3 to 13.
- **Locomotion**: Robots were either *stationary* or navigated using a *Brownian motion* gait.
- **Topology inference**: Peer transmission priority was governed by either a *stochastic* shuffle or a *comm-aware* ranking strategy based on link quality social evidence.
- **Message limits**: A *message limit* controlled how frequently agents could communicate with one another.
- **Transmission frequency**: Transmission bursts were optionally delayed by a random interval derived from the maximum observed peer latency.

A detailed description of the communication independent variables is available in Section IV-D.

A. Rastrigin Function

The Rastrigin function is defined as follows [6]:

$$f_R(\mathbf{x}) = 10n + \sum_{i=1}^{n} \left(x_i^2 - 10\cos(2\pi x_i)\right) \tag{1}$$

Where (n) is the number genes migrating between islands (robots). Whereas, (x) is the individual genome being evaluated. The function has a global minimum at $f_R(\mathbf{x}) = 0$. Early single-robot trials showed that $n = 10$ created enough search difficulty for performance to be measurable across swarm sizes.

B. Robot Platform

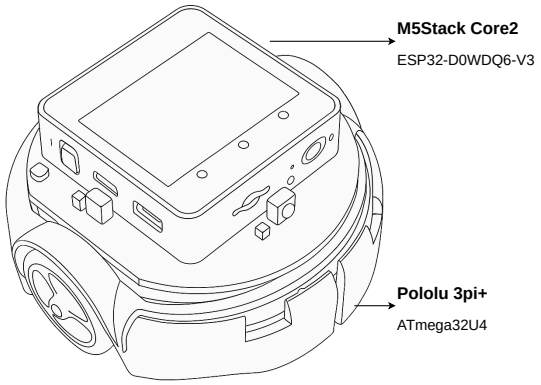


Fig. 2. The Swarm-B2 platform: M5Stack Core2 and Pololu 3Pi+

Table I summarizes the main hardware components of the Swarm-B2 platform used in this study [14]. Devices coordinate via an I^2C bus. The ESP32 microcontroller runs dual-core FreeRTOS tasks, that logs real-time data to a local SD card, and manages communication using the ESP-NOW data link (2.4 GHz).

TABLE I
SWARM-B2 HARDWARE STACK

Component	Interface	Function
M5Stack Core2 (240 MHz)	I^2C Master	Embedded evolution, ESP-NOW communication, data logging and user interface
Pololu 3Pi+ (16 MHz)	I^2C Slave	Locomotion using bumper and line following sensors

The Pololu 3Pi+ is equipped with a line following sensor array and two bump sensors, which were used to navigate the around arena as the robots have no positional sensing. A *Brownian-motion* gait was selected to maintain unbiased mobility across the arena and ensure constant movement. The gait code exposes the Pololu 3pi+ slave to the ESP32 master node, this interface lets the ESP32 act as a passenger with override, that can raise `START`/`STOP` flags. Figure 15 shows the on-board display, used only for debugging and device status during development. A real-time clock (RTC) synchronised experiment runs across the swarm.

IV. IMPLEMENTATION

Reliable and precise data logging is a pre-requisite for evaluating the communication performance and evolution dynamics of the swarm. To achieve this, our hardware-in-the-loop framework implements the following mechanisms to capture key evaluation metrics.

A. System Architecture

The firmware was implemented using the Espressif's ESP-IDF framework to provide real-time multitasking. Both, firmware updates and data capture are cloud enabled. Code that is written locally in VSCode (ESP-IDF v5.1.4) is rebuilt by GitHub for validation. Only binaries that pass tests are signed and uploaded to an Amazon Web Services (AWS) S3 OTA bucket with a versioned register. On boot, each robot fetches the version register, streams newer releases into a local OTA slot, and then resets with the latest firmware.

Experiment schedules ran in batches. During execution, time-stamped messages and event logs are written to a local SD card. After execution, the robot uses WiFi to upload the logs into an AWS S3. These files are serialised as JSON, tagged with an `experiment_id`, and later processed with AWS Glue/Athena and visualised in PowerBI. This data processing pipeline, allowed us to harvest high-volumes of traceable log data per-robot and per-experiment (see Section IV-E), while keeping the ESP-NOW data link isolated from Wi-Fi interference during experiments.

B. Embedded Evolution

The swarm uses a distributed genetic algorithm [14], to find the global minimum for Eq. 1. A visual representation of this is shown in Fig. 4. As the local population in each agent evolves, it communicates its local fitness and corresponding genome to its peers.

Our implementation employs an elitist migration strategy, this happens when the local GA reaches a patience threshold and the agent pushes its *best* (lowest fitness score) genome to other swarm members via ESP-NOW. The incoming remote genes from another peer are integrated into the local population by replacing the worst performing 5% individuals, this value was chosen to preserve genomic high fitness locally.

To avoid stagnation over local-minima, a mass extinction event together with a hyper-mutation mechanism tracks consecutive non-improving generations [15]. In each agent, if the local population's fitness remains stale for over 3 seconds, the mutation probability is temporarily increased and the lowest performing half of the population is re-initialised.

Evolution continues locally until the experiment ends, triggered either by (i) discovering the global optimum (fitness = 0) or (ii) a 60s timeout with no convergence. Two FreeRTOS tasks run in parallel, one executes the GA and the other handles ESP-NOW messaging.

C. Message Structure

We evaluate robot-to-robot communication with a fixed ESP-NOW message schema (Table II), floats are rounded to $3d.p.$ to send the message in a single transmission. The implication of this design choice is discussed in Section VI.

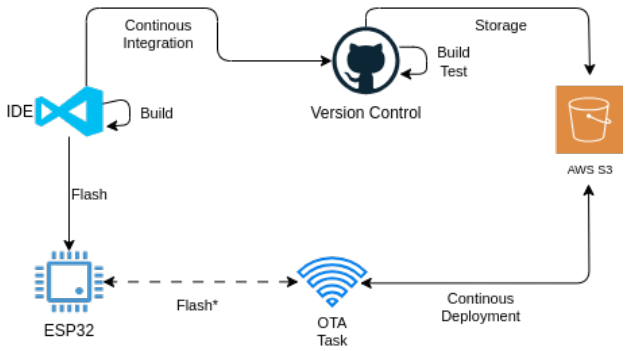


Fig. 3. Integrated architecture: Local development, GitHub repository, AWS S3 storage, and OTA updates.

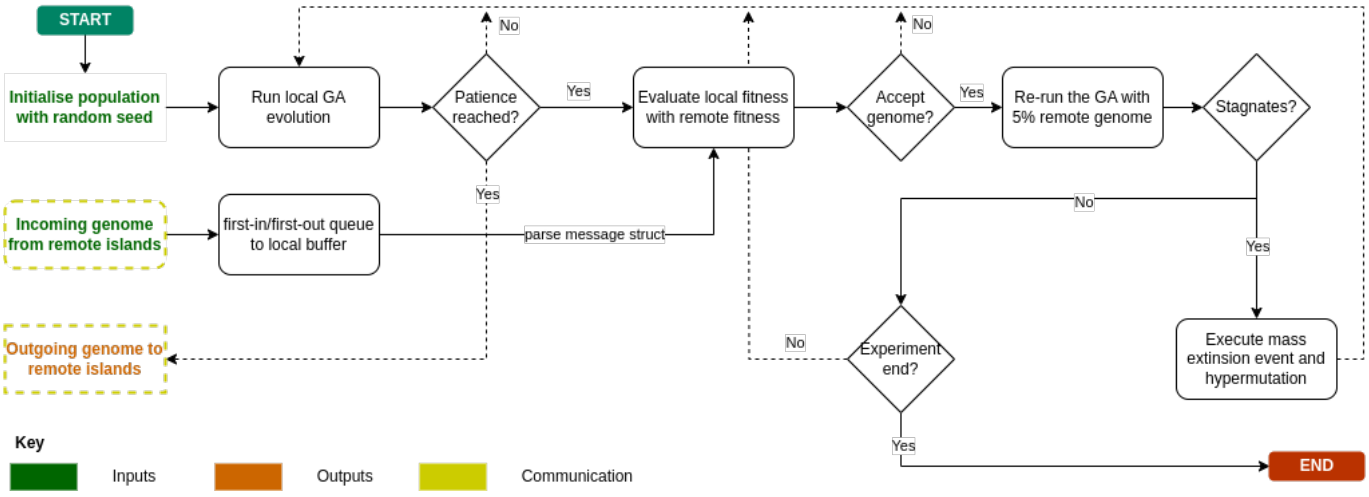


Fig. 4. Distributed island-model GA executed on the ESP32 swarm

TABLE II
MESSAGE STRUCTURE USED FOR ESP-NOW FITNESS VALUE AND GENOME EXCHANGE

Field	Type	Description
`log_id`	uint32_t	Unique internal identifier of the event.
`robot_id[5]`	char	MAC identifier for the sender robot plus a null terminator.
`log_datetime`	time_t	Timestamp based on the internal RTC.
`message[128]`	char	Content of the message including the fitness value and the 10-gene genome delimited with "|".

D. Communication Layer

In contrast to previous work by [13], that explored decentralised communication protocols using middleware. We exploit the ESP32's Medium Access Control (MAC) layer to keep packet timing and load directly measurable. To do this, we implement ESP-NOW by pre-registering the MAC addresses of all peers in each agent's firmware. We do not view this as a hard scalability limit in the swarm paradigm, as [16] explains that gossip-style OTA propagation can disseminate firmware in swarms, allowing MAC registers to scale without central coordination.

Our design uses push only event-driven unicast messaging, where each device transmits data to peers without waiting for requests. Using this scheme avoids the complexity of pull-based communication protocols. Incoming messages are handled by a first-in/first-out queue and buffered locally for evaluation. This allowed internal task operations to continue uninterrupted, at the cost of internal memory, from pilot experiments we set the queue size to 40 (≈8KB).

Social evidence, in our case is present in the form of the link-quality that arises from migration. Specifically from the latency and Received Signal Strength Indicator (RSSI) that a peer perceives relative to other peers. We use this social evidence to inform our communication strategy and adapt the following policies.

1) Topology Inference

We investigate the impact of two communication schemes. These were designed to influence the priority and order with which each agent sends messages. We use the social evidence retained locally as a proxy to infer the topology, under the assumption that distant islands will exhibit different link-quality characteristics.

These are implemented as follows:

- *Stochastic*: Each agent calls a random seed to apply a Fisher-Yates shuffle [17] over the list of peer MAC addresses, sorting them before sending messages to *all peers*. No social evidence bias is used, every order permutation is equally likely.
- *Comm-aware*: Each agent ranks its peers based on the most recent measurements of latency and RSSI. Peers with unknown metrics (at initialisation) are prioritised first to ensure social evidence is available. Then, peers are scored by normalizing both latency and RSSI, and those only in the *worst half* (highest latency, lowest RSSI) are messaged.

For *stochastic*, the shuffle is defined in Eq. 2, where (j_i) is a random index drawn from a uniform distribution. (σ) is the random permutation of MAC addresses for each (i) peer. We can think of this scheme as a *pseudo-broadcast* as the genome is sent to all peers with negligible delays between transmissions.

$$\sigma = \prod_{i=n}^{2}(i, j_i), \qquad j_i \sim \text{Uniform}\{1, \ldots, i\} \tag{2}$$

Whereas for *comm-aware*, the scoring (S) per-peer (i) is defined in Eq. 3. Here, the round-trip latency (l) can be measured via acknowledgements (ACKS). The RSSI (r) is

reported by the ESP-NOW send callback after each successful transmission. If multiple messages have been sent, only the most recent metrics stay in memory. Note that both RSSI and latency are scored equally.

$$S_i = \frac{l_i - l_{\min}}{l_{\max} - l_{\min}} + \frac{r_{\max} - r_i}{r_{\max} - r_{\min}} \tag{3}$$

2) Message Limits

Inspired by the reported *less-is-more* effects [10], we cap the number of migration messages that each robot may send (Eq. 4). Each agent is given a message budget (B) per sliding time window (W). Once the limit is exhausted, the robot stays silent until the window advances, no matter how often its onboard genetic algorithm requests migration. Different from earlier IR-based systems that relied on a short-range *narrowcast* channel [10], we deliver the message to peers using the topology inference schemes discussed previously.

$$\forall t: \quad M_W(t) = \sum_i \mathbf{1}_{\{t-W<t_i\le t\}} \le B \tag{4}$$

Here, (t_i) are the send times, $M_W(t)$ is the number of broadcast messages sent in the last $W = 8$s, and the limit is fixed at $B = 1$. These values were chosen after pilot tests showed that, without the limit, each peer emitted roughly 15 messages per experiment, effectively halfving migration.

3) Transmission Frequency

To further explore the communication behaviour of the swarm under simultaneous transmission conditions, we modulate the migration interval between islands to spread transmission bursts. Each migration is postponed by random delay drawn from the highest peer-to-peer latency observed so far, which is estimated from the latest ACK round-trip times on every link, described by Eq. 5:

$$\Delta t \sim \mathrm{Uniform}\big(0,\, L_{\max}\big), \qquad L_{\max} = \max_i\{\tau_i\} \tag{5}$$

where (τ_i) is the one-hop latency to peer (i). If random mode is disabled, ($\Delta t = 0$) and messages are sent immediately by calling `esp_now_send`. The intention of this approach is to control the transmission of messages in a stochastic manner to reduce unintended channel contention in the swarm network.

E. Performance Metrics

To evaluate the quality of the communication in the swarm, each robot records granular *sender-receiver* logs: latency (l) ms, RSSI (r) dBm, net kilobyte rate (t) Kbps, transmission-error flags (e), and per-core CPU load (u). These raw logs are post-processed by the data pipeline previously detailed in Section IV-A. AWS computes the aggregated per-device and per-experiment metrics: (L_m) *mean latency* from ACKs, (J_m) *mean jitter*, the standard deviation of successive latencies, (E_m) *mean error rate*, the percentage of messages that were sent but not acknowledged, and (T_m) *mean throughput*, the data rate successfully transferred by each agent. With (F_m) *mean fitness value* of the GA and (A_m) *mean adaptation rate* defined as the cumulative percentage of peers sharing the lowest F_m over a 2s window.

To assess overall network performance, a QoS score was computed as:

$$\mathrm{QoS} = w_0 \cdot (1-\hat{L}) + w_1 \cdot (1-\hat{J}) + w_2 \cdot (1-\hat{E}) + w_3 \cdot \hat{T} \tag{6}$$

Where, $\hat{L}$, $\hat{J}$, $\hat{E}$, $\hat{T}$ are the normalised global metrics aggregated at the experiment-level in the range of $[0, 1]$ and the weights w_i are defined in Table III. A higher QoS value indicates better overall network communication performance. QoS_c weights latency/jitter for consensus/voting tasks, QoS_s weights errors and throughput for sparse deployments.

TABLE III
QoS UTILITY FUNCTION WEIGHTS FOR DIFFERENT SWARM ROBOTICS APPLICATIONS

QoS	w_0	w_1	w_2	w_3	**Application**
QoS_c	0.5	0.25	0.15	0.1	Swarm consensus and voting
QoS_s	0.15	0.15	0.50	0.2	Swarm sparse deployments

No inferential statistics were performed. For each condition we executed $n \ge 16$ independent runs. Figures report median with IQR (or mean ± SD where noted). All comparisons are descriptive and should be interpreted accordingly. Experiments used fixed per-run random seeds for GA initialisation. Collected logs and analytic scripts are archived in [18], in the `data_analysis` folder.

V. RESULTS

As shown in Fig. 5a, the latency and jitter of the swarm network differ significantly between topology inference schemes. Across experiments, *comm-aware* generally yeilds lower latencies and more stable delivery compared to the *stochastic* mode. Fig. 5b-c show throughput vs topology/message limits. *Stochastic* yields higher raw throughput with wider variability.

Larger swarms reach lower mean fitness scores. *Stochastic* schemes accelerate information diffusion, where ≈80% of agents share the temporal "best" solution 20 seconds earlier than under *comm-aware*. Fig. 6a-b. The degree of connectivity per-peer across time shows *stochastic* forms more links compared to *comm-aware*, both increasing over time. Fig. 7.

Error rates are small and comparable for smaller density swarms, rising during 13-agent swarm experiments and when the network operates under the *stochastic* scheme. Fig. 8a-b. With regards to locomotion Fig. 9a shows that Brownian motion results with slightly improved RSSI values, indicating more stable connections compared to static swarms. Whereas, Fig. 9b suggests no differences between RSSI and topology inference schemes.

TABLE IV
EXPERIMENT SCHEDULE AND MEAN RESULTS ± SD, MINIMUM OF 16 RUNS PER-EXPERIMENT

Robots	Topology	Locomotion	Message Limit	Transmission	L_m(ms)	J_m(ms)	E_m(%)	T_m(Kbps)	QoS_c	QoS_s
3	Stochastic	Static	Unlimited	None	15.1±1.9	10.8±2.1	0.0±0.0	0.3±0.8	0.86	0.82
			Unlimited	Modulated	20.8±3.5	10.0±1.8	0.1±0.05	0.3±0.8	0.83	0.81
			Limited	None	12.9±2.7	8.6±2.4	0.0±0.0	0.2±0.6	0.85	0.80
		Brownian	Unlimited	None	13.0±1.8	9.3±1.6	0.0±0.0	0.3±0.8	0.87	0.83
			Unlimited	Modulated	17.7±2.9	9.5±2.5	0.1±0.05	0.3±0.8	0.84	0.82
			Limited	None	11.1±2.3	7.3±2.0	0.0±0.0	0.2±0.6	0.87	0.81
	Comm-aware	Static	Unlimited	None	11.1±1.8	7.6±2.7	0.0±0.0	0.2±0.5	0.87	0.80
			Limited	None	10.5±1.7	6.9±4.0	0.0±0.0	0.1±0.4	0.86	0.79
		Brownian	Unlimited	None	9.0±1.6	5.9±2.3	0.3±0.02	0.2±0.5	0.87	0.77
			Limited	None	8.2±1.7	4.5±2.2	0.0±0.0	0.1±0.3	0.88	0.79
8	Stochastic	Static	Unlimited	None	43.5±14.6	23.7±4.6	0.1±0.01	0.7±2.6	0.72	0.84
			Unlimited	Modulated	37.5±10.4	22.2±7.3	0.1±0.01	0.6±2.5	0.75	0.84
			Limited	None	31.1±15.2	14.7±5.3	0.1±0.01	0.4±1.8	0.78	0.82
		Brownian	Unlimited	None	32.7±13.2	20.7±4.6	0.2±0.01	0.6±2.5	0.77	0.85
			Unlimited	Modulated	40.0±9.4	21.0±5.9	0.1±0.01	0.6±2.4	0.77	0.85
			Limited	None	25.6±13.5	12.8±5.1	0.2±0.01	0.4±1.7	0.81	0.82
	Comm-aware	Static	Unlimited	None	19.4±6.3	13.3±3.0	0.04±0.01	0.4±1.2	0.84	0.84
			Limited	None	18.7±6.7	11.3±3.3	0.1±0.01	0.3±0.9	0.83	0.80
		Brownian	Unlimited	None	15.3±5.6	10.2±2.5	0.1±0.01	0.4±1.7	0.87	0.84
			Limited	None	15.0±6.4	8.9±3.5	0.1±0.02	0.3±0.9	0.85	0.80
13	Stochastic	Static	Unlimited	None	91.5±33.4	47.4±8.4	1.8±0.7	0.77±3.7	0.37	0.58
			Unlimited	Modulated	51.2±20.2	33.4±13.0	0.2±0.1	0.72±3.0	0.64	0.81
			Limited	None	56.0±33.8	22.6±9.5	1.6±0.5	0.58±2.7	0.61	0.65
		Brownian	Unlimited	None	86.3±34.0	45.4±9.2	1.9±0.7	0.8±3.7	0.40	0.58
			Unlimited	Modulated	51.7±18.3	35.0±12.8	0.3±0.1	0.7±3.1	0.64	0.80
			Limited	None	53.1±30.3	21.9±8.3	2.4±0.7	0.6±2.6	0.60	0.57
	Comm-aware	Static	Unlimited	None	33.5±16.9	17.3±5.4	0.2±0.02	0.6±2.0	0.78	0.83
			Limited	None	27.5±16.5	11.8±5.8	0.2±0.01	0.4±1.5	0.80	0.81
		Brownian	Unlimited	None	31.5±15.3	16.5±4.1	0.7±0.03	0.6±2.0	0.77	0.79
			Limited	None	26.3±15.5	11.1±4.7	0.8±0.03	0.4±1.5	0.79	0.75

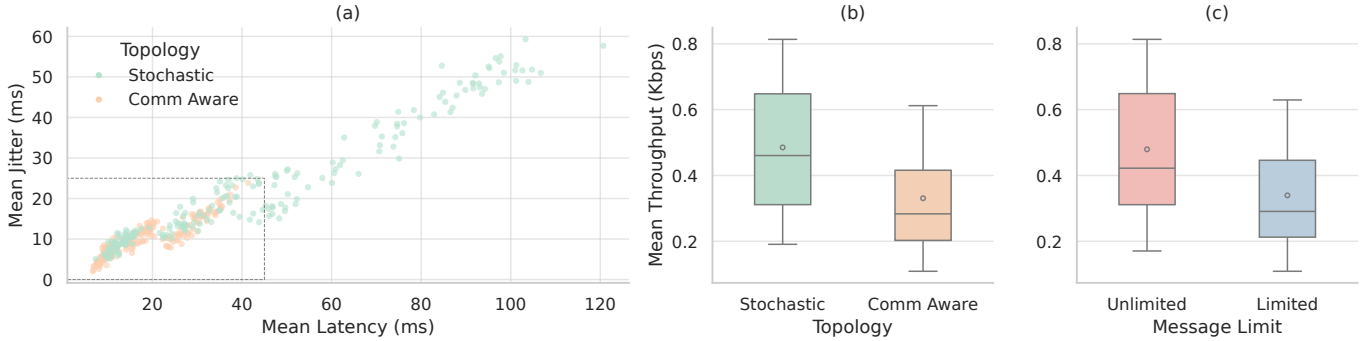


Fig. 5. (a) Latency and Jitter relationship under varying topology schemes, (b)(c) throughput distribution under topology and message limit variables

Adding random bounded-delays during transmission lightly stabilises latency over time with negligible throughput loss, albeit mean jitter and latency values increase compared to the *comm-aware* scheme (Fig. 10a-c). The transmissions delays do not block convergence and achieve comparable adaptation rates compared to the *stochastic* scheme (Fig. 11a-b).

An evaluation of the characteristics of the QoS metrics for the 13-agent swarm, show that (i) *comm-aware* achieves high QoS_c scores, (ii) modulated transmission shifts toward QoS_s but scores less in QoS_c, (iii) message limits do not have a clear effect on QoS, both kernel densities overlap each other. Fig. 12a-d.

VI. DISCUSSION

Across conditions, lower end-to-end latency is driven primarily by limiting concurrent links, not by message limits, achieving sub-50ms networks comparable to [7] for swarm communication relay scenarios. Under *stochastic* scheduling conditions, rapid increases in unique connections raise contention. *Comm-aware* limits link formation, avoiding burst traffic and yielding lower median latency and jitter at throughput cost. Time-snapshots of inferred topologies in Fig. 13 illustrate this mechanism, with *stochastic* creating denser transient link sets than *comm-aware* over comparable intervals. In our elitist island-model setup, slower mixing between islands can be useful. Limiting link formation in

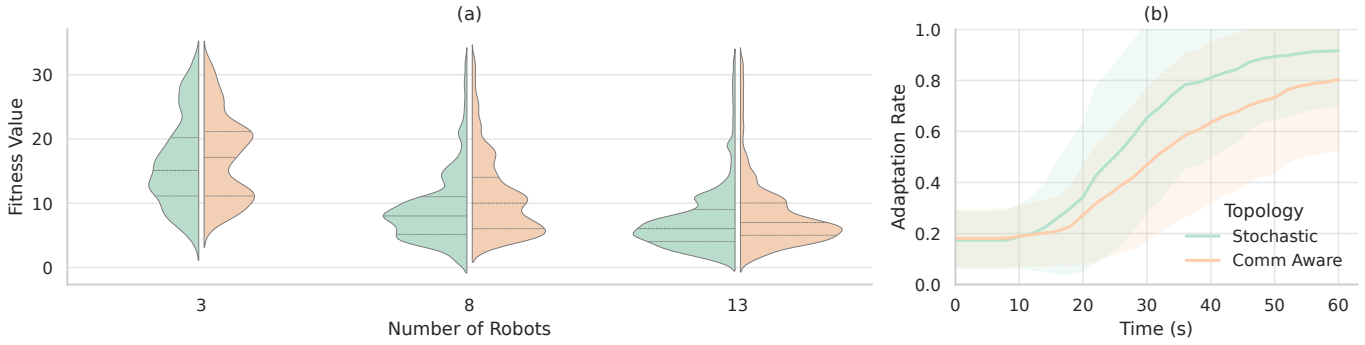


Fig. 6. (a) Mean device fitness values across different swarm sizes, (b) mean cumulative percentage of agents converging to the lowest fitness value across $2s$ windows

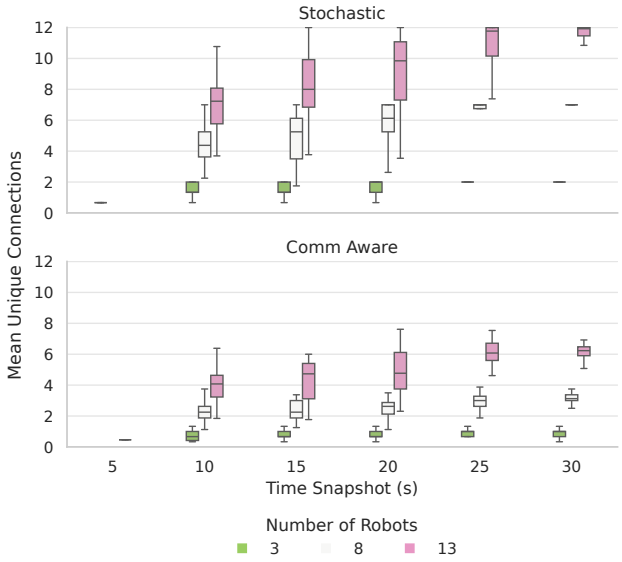


Fig. 7. Per-peer number of mean connections formed by time snapshot up to 30 seconds for both topology inference schemes

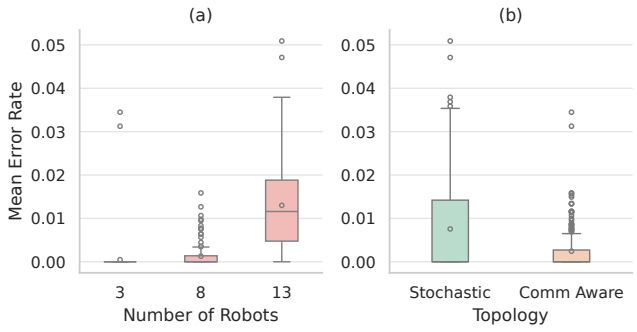


Fig. 8. (a) Mean error rates by device across varying swarm sizes, (b) mean error rate for different topology inference schemes

comm-aware reduces premature convergence, allowing for a more diverse genome to persist and be explored longer. This aligns with the observation that overall fitness improves primarily with agent count (Fig. 6a), while the fastest

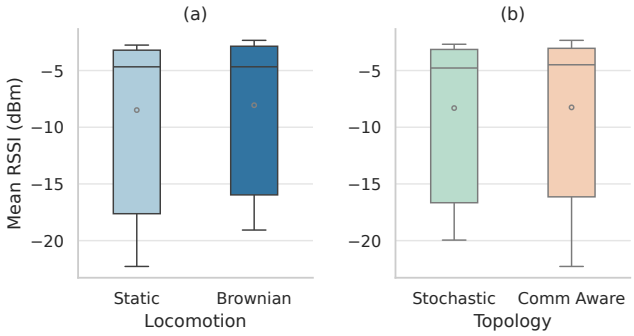


Fig. 9. (a) Mean RSSI values across different locomotion modes, (b) mean RSSI values across different topology inference schemes

adaptation rates arise under the *stochastic* scheme where information diffuses quickly (Fig. 11b).

Comm-aware attains consistently high QoS values by suppressing jitter and keeping low latencies, while *stochastic* yields higher raw throughput but with higher error rates as agent density grows. Kernel-density plots of QoS_c against QoS_cs in Fig. 12a-d, shows *comm-aware* achieves an overall improved service relative to the overall distribution, whereas frequency modulated *stochastic* shifts the distribution toward QoS_s. Message limits show no clear effect.

Error rate increases with swarm size, consistent with higher collision probability (Fig. 8a-b). This effect is independent of locomotion, and becomes most pronounced under *stochastic* at 13-robot swarms. RSSI does not identify topology schemes in this arena, which means that the *comm-aware* scheme is effectively ranking peers by latency. Brownian motion yields slightly lower RSSI than static (Fig. 9a-b), plausibly by incidental repositioning into stronger links. Verifying this, would require sparser layouts in larger arenas.

Introducing frequency modulation under the *stochastic* scheme smooths latency over time (Fig. 10b) without impacting throughput, and preserves fast adaptation rates

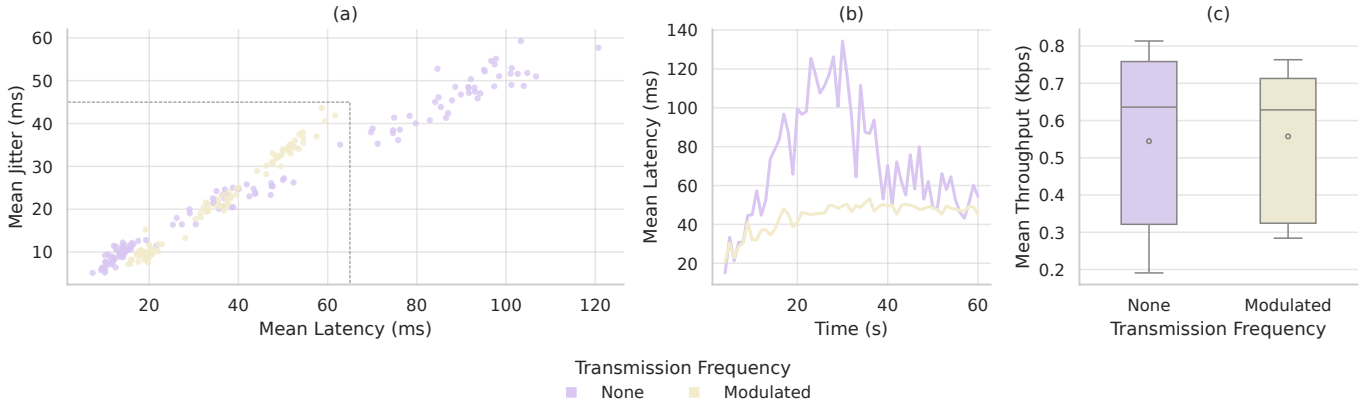


Fig. 10. (a) Latency and Jitter relationship under varying transmission policies, (b) latency across time by transmission setting, (c) throughput distribution by transmission setting

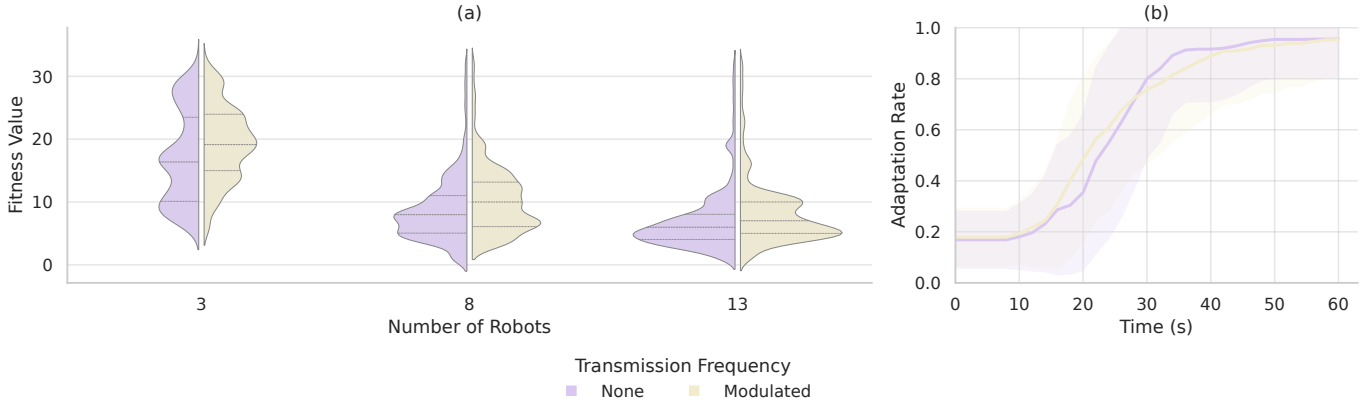


Fig. 11. (a) Mean device fitness values across different swarm sizes, (b) mean cumulative percentage of agents converging to the lowest fitness value across $2s$ windows

relative to *comm-aware*. Validating the findings of [11], the swarm can tolerate transmission delays and attain similar fitness scores compared to having no modulation. For tasks where rapid environmental updates matter such as search and rescue [8], this policy provides a low complexity alternative to flooding.

Using the fixed genome structure (Table II) in a single-channel setup, we observed no detectable effects of mean swarm throughput on GA performance. However, we have not tested whether alternative payload schemas (e.g., partial-genome transfer or non-elitist migration) could change this relationship. The generality beyond the ESP-NOW's 250-byte packet limit also remains unverified, larger messages that require fragmentation would likely complicate asynchronous processing, requiring deeper local buffers to avoid packet loss, and also increase compute overhead.

Energy consumption matters. Broadcast-heavy policies raise CPU utilisation, whereas link-constraining policies lower processing load (Fig. 14). Energy budgets therefore favour selective link formation and moderated transmissions. We estimate that at a constant load of 200mW, we could extend battery life of the system by ≈33s every hour (1% CPU$u \approx 0.74$ mWh/h) using the *comm-aware* scheme.

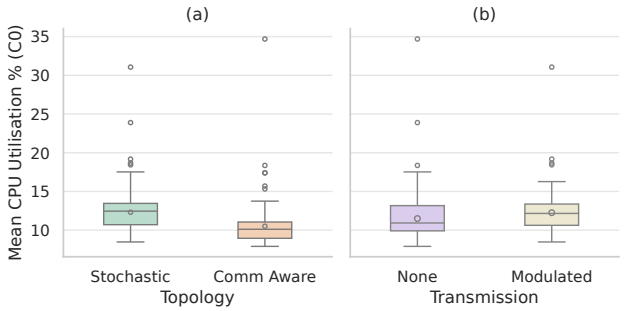


Fig. 14. Core 0 utilisation by communication policy, excluding message limited experiments

For context, ESP-NOW transmit currents are typically an order of magnitude higher than Bluetooth, so depending on the application swarms may need to prioritise partial link

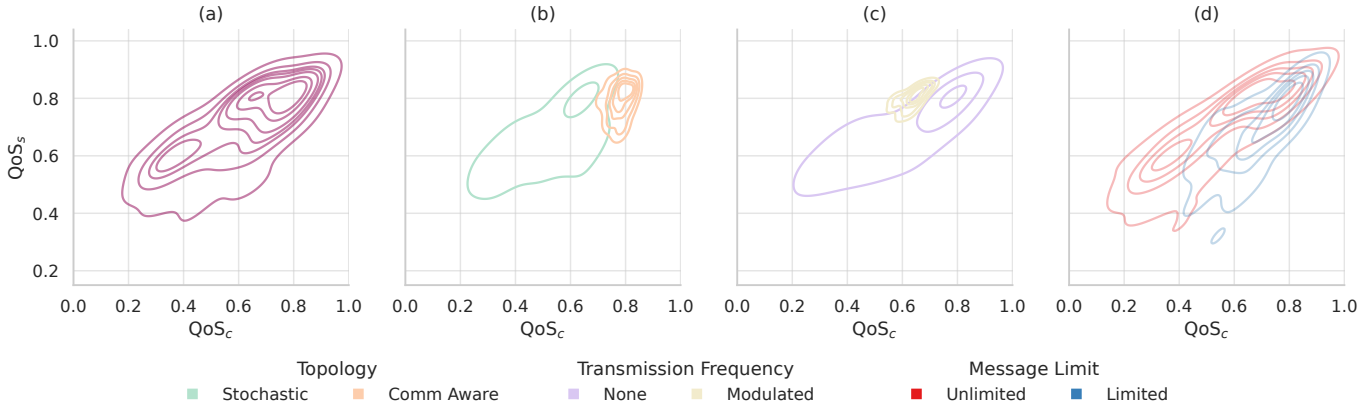


Fig. 12. Kernel-density contours showing the joint distribution of two QoS metrics (x: QoS_c, y: QoS_s) for a 13-agent swarm. (a) pooled across all experimental factors, (b) split by communication topology, (c) split by transmission frequency policy, (d) split by message limit setting

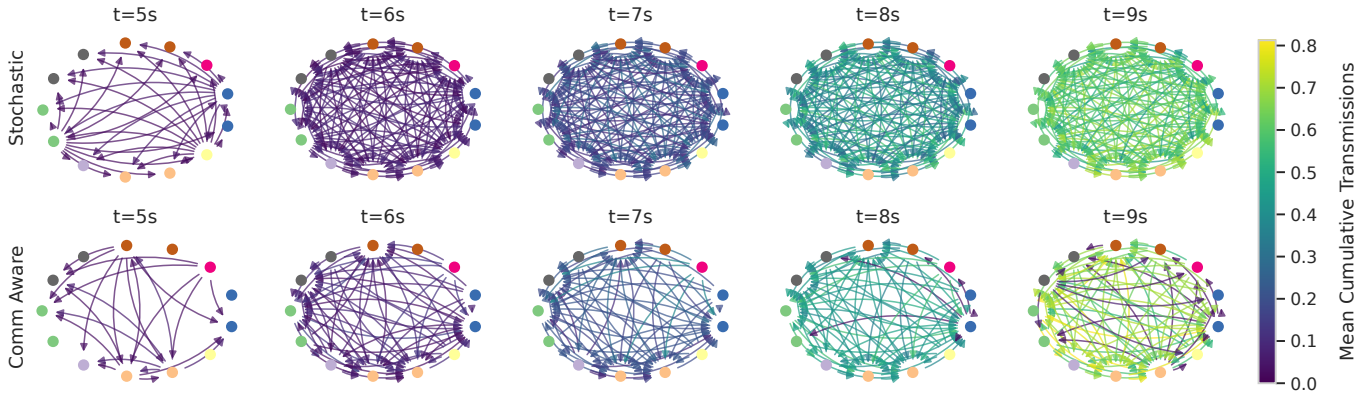


Fig. 13. Temporal snapshots of inferred communication topologies for a 13-agent swarm. Rows: inference scheme, columns: time samples. Nodes are on a fixed circular layout, edges show inferred links and its colour indicates mean cumulative transmissions per node.

formation over fast information diffusion. This trade-off is not abstract, in the author's current role at Unilode Aviation Solutions (the operator of the largest globally distributed aviation IoT network), battery longevity outranks other QoS metrics due to the economics of battery replacement.

Answers to the research questions: **RQ1**: Link-aware peer ordering abates latency and jitter by trimming concurrent links between peers, at a small throughput cost where message limits matter less and the effects are more noticeable in swarms with more than 8-agents (Fig. 5a-c; Fig. 13). **RQ2**: *Stochastic* ordering accelerates diffusion and supports faster convergence, larger swarms achieve lower mean fitness throughout independent variables and irrespective of throughput (Fig. 6a-b). **RQ3**: Latency-bounded transmission delays smooths latency variation during message bursts with negligible throughput loss and preserve adaptation rates (Fig. 10b-c; Fig. 11b).

A. Limitations

RTC drift (roughly $\pm$1s) impacts time resolution, so message timestamps are only loosely synchronised. A second limitation is the environmental variability. Although all trials ran in the same facility, ambient Wi-Fi activity may have varied. We set the ESP-NOW link to channel 6 to avoid interference from nearby access points, but environmental noise cannot be entirely ruled out. Results should therefore be interpreted with these limitations in mind.

VII. Conclusion

This work profiled peer-to-peer communication for robot swarms on ESP32 hardware. Across 3 to 13 robot swarms, we found that link formation and transmission timing, drive QoS and adaptation rates on this platform, with throughput volume being secondary. Link formation schemes based on latency social evidence, reduced adaptation rates and information diffusion but did not prevent convergence. Conversely, introducing a small, randomised per-packet transmission delay bounded by the recent peer latency, lightly stabilised network latency without materially impacting throughput.

These findings translate into two pragmatic design rules for ESP32 swarms with more than 8-peers:

- When rapid network-wide diffusion and high throughput are the priority (e.g., swarm environmental updates or consensus), prioritise *stochastic* scheduling with randomised bounded transmission delays.
- When preventing premature convergence and achieving low latencies is of concern (e.g., sparse deployments), focus on limiting effective link formation to lower contention and support diversity in evolutionary search.

This study supports the *less-is-more* effects observed in prior swarm studies [10], where well-scoped constraints can be a feature in decentralized communication.

A. Future Work

Future work will stress-test larger, dynamically varying payloads to map the limits of our conclusions and determine whether the same policies extend to other hardware configurations (e.g., IR, Bluetooth) and to multi-channel platforms. Finally, leveraging the OTA update mechanism with an evolutionary coding agent such as *OpenEvolve* [19], could enable novel swarm meta-evolution frameworks. These could support the fine-tuning of communication policies (and embedded evolution parameters) for specific applications, such as long-term swarm deployments that optimise for energy efficiency.

VIII. Appendix

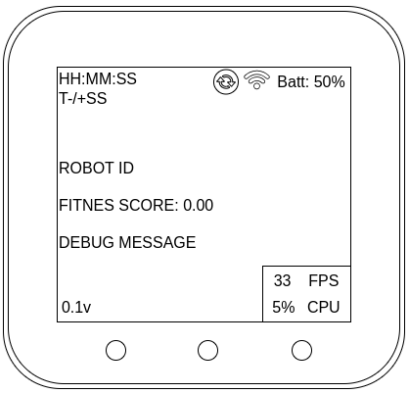


Fig. 15. On-board debug/status display

References

[1] Heiko Hamann. *Swarm Robotics: A Formal Approach*. Cham: Springer International Publishing, 2018.

[2] Marco Dorigo et al. "Ant algorithms and stigmergy". In: *Future Generation Computer Systems* 16.9 (June 1, 2000). MAG ID: 2151758915, pp. 851–871.

[3] Tan Jian Ding et al. "Advancements and Challenges of Information Integration in Swarm Robotics". In: *2023 IEEE International Conference on Cybernetics and Intelligent Systems (CIS) and IEEE Conference on Robotics, Automation and Mechatronics (RAM)*. 2023 IEEE International Conference on Cybernetics and Intelligent Systems (CIS) and IEEE Conference on Robotics, Automation and Mechatronics (RAM). ISSN: 2326-8239. June 2023, pp. 89–95.

[4] Xing An et al. "Multi-Robot Systems and Cooperative Object Transport: Communications, Platforms, and Challenges". In: *IEEE Open Journal of the Computer Society* 4 (2023). Conference Name: IEEE Open Journal of the Computer Society, pp. 23–36.

[5] Motoaki Hiraga et al. "When Less Is More in Embodied Evolution: Robotic Swarms Have Better Evolvability with Constrained Communication". In: *Journal of Robotics and Mechatronics* 35.4 (Aug. 20, 2023), pp. 988–996.

[6] M. Ruciński, D. Izzo, and F. Biscani. "On the impact of the migration topology on the Island Model". In: *Parallel Computing*. Parallel Architectures and Bioinspired Algorithms 36.10 (Oct. 1, 2010), pp. 555–571.

[7] Sabine Hauert, Jean-Christophe Zufferey, and Dario Floreano. "Evolved swarming without positioning information: an application in aerial communication relay". In: *Autonomous Robots* 26.1 (Jan. 1, 2009), pp. 21–32.

[8] Dimitri Perrin and Hiroyuki Ohsaki. "Decentralised Communication in Autonomous Agent Swarms". In: *2012 26th International Conference on Advanced Information Networking and Applications Workshops*. 2012 26th International Conference on Advanced Information Networking and Applications Workshops. Mar. 2012, pp. 1143–1146.

[9] Mohamed S. Talamali et al. "When less is more: Robot swarms adapt better to changes with constrained communication". In: *Science Robotics* 6.56 (July 28, 2021). Publisher: American Association for the Advancement of Science, eabf1416.

[10] Till Aust et al. "The Hidden Benefits of Limited Communication and Slow Sensing in Collective Monitoring of Dynamic Environments". In: *Swarm Intelligence*. Ed. by Marco Dorigo et al. Lecture Notes in Computer Science. Cham: Springer International Publishing, 2022, pp. 234–247.

[11] Konstantinos I. Tsianos and Michael G. Rabbat. *The Impact of Communication Delays on Distributed Consensus Algorithms*. July 24, 2012.

[12] Stefan M. Trenkwalder et al. "SwarmCom: an infra-red-based mobile ad-hoc network for severely constrained robots". In: *Autonomous Robots* 44.1 (Jan. 1, 2020), pp. 93–114.

[13] Mahmoud Almostafa Rabbah et al. "Real Time Distributed and Decentralized Peer-to-Peer Protocol for Swarm Robots". In: *International Journal of Advanced Computer Science and Applications (IJACSA)* 12.11 (Jan. 30, 2021). Number: 11 Publisher: The Science and Information (SAI) Organization Limited.

[14] *University of Bristol: Dissertation Guidance*. In collab. with Paul O'Dowd. June 1, 2025.

[15] T. Krink and R. Thomsen. "Self-organized criticality and mass extinction in evolutionary algorithms". In: *Proceedings of the 2001 Congress on Evolutionary Computation (IEEE Cat. No.01TH8546)*. 2001 Congress on Evolutionary Computation. Vol. 2. Seoul, South Korea: IEEE, 2001, pp. 1155–1161.

[16] Vivek Shankar Varadharajan et al. "Over-the-Air Updates for Robotic Swarms". In: *IEEE Software* 35.2 (Mar. 2018), pp. 44–50.

[17] Ronald Aylmer Fisher and Frank Yates. *Statistical tables for biological, agricultural, and medical research*. 6th ed., rev. and enl. OCLC: 550706. New York: Hafner Pub. Co, 1963.

[18] Luis Yallico. *https://github.com/yallico/robotics_dissertation*. June 1, 2025.

[19] Asankhaya Sharma. *https://github.com/codelion/openevolve*. Aug. 1, 2025.